AAP + SPIFFE/Vault
Zero-Trust Deep Dive

Replacing static Vault credentials with cryptographic workload identity for Ansible Automation Platform. Covers architecture, RBAC model, cluster verification, non-K8s execution, Vault Enterprise, and granular isolation.

October 2026 | POC on hosted-1 (HyperShift KubeVirt, OCP)

1. Why This Matters

AAP natively supports several Vault auth methods (AppRole, scoped tokens, Kubernetes auth), so a root token is not strictly required. However, the layered-zero-trust validated pattern wired AAP with the root token, and many customer deployments default to a single long-lived token or AppRole secret_id stored in AAP's database.

Even with AppRole or K8s auth, you still have stored credentials (a secret_id or a service-account-scoped token), and identity is flat: every job running under the same SA or AppRole gets the same Vault access regardless of which AAP organization or playbook triggered it.

Approach	Stored Secret	Rotation	Scope	Identity Proof
Root token (validated pattern)	Root token in AAP DB	Manual	Everything	None
AppRole (native AAP)	secret_id in AAP DB	Manual or wrapped	Policy-scoped	Shared secret
K8s auth (native AAP)	Nothing (SA token)	Automatic	Policy-scoped	Pod SA token
SPIFFE JWT (this POC)	Nothing	Automatic	Policy-scoped, per-job	Cryptographic attestation

Why SPIFFE, not just AppRole or K8s auth? AppRole and K8s auth already eliminate the root token. SPIFFE adds three things: (1) identity is attested by the kubelet, not just asserted by a bearer token; (2) per-job granularity via pod labels, so different org EE pods get different identities even with the same service account; (3) portable trust that federates across clusters.

2. Architecture

Namespace	Component	Type	Role
spire-system	SPIRE Server + OIDC sidecar	StatefulSet	Trust root, CA, OIDC key server
spire-system	SPIRE Agent	DaemonSet	Workload API on every node
vault	Vault	StatefulSet	Secret storage, JWT auth
kyverno	Kyverno	Deployment	Sidecar injection policy
aap	AAP 2.7	Operator-managed	Controller, Hub, EDA, Gateway

OIDC Discovery Provider: runs as a sidecar in the SPIRE Server pod. It publishes SPIRE's public signing keys as standard JWKS at /.well-known/openid-configuration. Vault fetches these keys to verify JWT-SVIDs. This is NOT the same as AAP Gateway's OIDC, which serves Gateway's own keys for human user SSO. They publish different key material for different purposes.

How Vault trusts SPIRE: Vault's JWT auth is configured with oidc_discovery_url pointing to the SPIRE OIDC Discovery Provider's internal K8s service URL. Vault periodically fetches the JWKS and uses the public keys to verify JWT-SVIDs locally. No direct SPIRE integration is needed; they communicate via standard OIDC.

3. End-to-End Flow

1. Kyverno intercepts AAP pod creation, injects spiffe-helper + spiffe-vault-client as init containers and sidecars
2. SPIRE Agent validates the pod via kubelet k8s_psat attestor
3. init-spiffe-helper fetches a JWT-SVID (5-min TTL) from the Workload API and writes it to /svids/jwt.token
4. fix-jwt-perms init container runs chmod 644 (spiffe-helper writes as root:root 0600, EE pods run as UID 1001)

5. init-spiffe-vault-client POSTs the JWT to Vault /v1/auth/jwt/login with role=aap-platform

6. Vault validates JWT signature against SPIRE's OIDC keys, checks sub and aud claims, returns scoped token (1h TTL)

7. init-spiffe-vault-client reads secrets, writes to shared volume, exits

8. AAP containers start and consume credentials; sidecars continuously rotate JWT and refresh secrets

4. RBAC Model (Four Layers)

Layer	Mechanism	What It Enforces
1. SPIRE selectors	k8s:sa: / k8s:pod-label:	Each pod type gets exactly one SPIFFE ID
2. Vault bound_subject	JWT sub must match role	JWT for org A can't auth to org B's role
3. Kyverno + Container Groups	Pod workload sidecar/init config	Controller gets platform role, EE gets org role
4. Lookup plugin	Per-call Vault auth	No ambient credentials, no pre-fetched cache

The multi-tenant tension: AAP's controller-task pod serves all orgs, but SPIFFE identity is per-pod. Resolution: the controller gets the platform SPIFFE identity (reads platform secrets), while per-org isolation happens at the EE layer via Container Groups with org-specific pod labels.

5. Playbook Consumption: spiffe_vault Lookup Plugin

The spiffe_vault Ansible lookup plugin authenticates to Vault per-lookup using the EE pod's JWT-SVID. Zero external dependencies (stdlib only). Each call reads the JWT file, POSTs to Vault /v1/auth/jwt/login, fetches the secret, and returns the value.

```
cloud_password: "{{ lookup('spiffe_vault',
'secret/data/aap/org/infra-corp/credentials', field='cloud_password',
vault_role='aap-org-infra-corp') }}"
```

6. Cluster Verification (October 2026)

All 5 E2E tests passed on the live hosted-1 cluster. Test pod spiffe-e2e-test was created in the aap namespace with label spiffe.io/spiffe-id=org-infra-corp to simulate an EE pod.

#	Test	Result
1	JWT-SVID has correct sub claim (org-infra-corp SPIFFE ID)	PASS
2	Own-org Vault auth + secret read (infra-corp credentials)	PASS
3	Cross-org auth denied (infra-corp JWT to appdev-squad role)	PASS: invalid subject (sub) claim
4	Cross-role auth denied (infra-corp JWT to platform role)	PASS: invalid subject (sub) claim
5	Cross-org secret read denied (infra-corp token reads appdev-squad path)	PASS: 403 Forbidden

Bug found and fixed: spiffe-helper writes jwt.token as root:root 0600, but EE pods run as UID 1001. Fixed with a fix-jwt-perms init container (chmod 644) after spiffe-helper.

7. Beyond Kubernetes: VM Execution Nodes

AAP supports hybrid execution. On VM execution nodes, receptor launches EE containers using podman. SPIRE's docker workload attestor works with podman containers and can select on container labels, enabling the same per-org isolation model as K8s.

The gap: AWX currently has no per-Instance-Group mechanism to inject custom podman labels (container_options_override). A small upstream PR (one model field + ~4 lines) would mirror the existing pod_spec_override pattern for K8s Container Groups.

Platform	Node Attestor	Workload Attestor
AWS EC2	aws_iid (Instance Identity Document)	docker (container labels)
Azure VM	azure_msi (Managed Service Identity)	docker (container labels)
GCP	gcp_iit (Instance Identity Token)	docker (container labels)
Bare metal	tpm_devid (TPM 2.0)	docker or unix (UID/binary path)

8. Vault Enterprise 1.21: SPIFFE Without SPIRE

Vault Enterprise 1.21 (GA March 2026) can act as a full SPIFFE server. The secrets/spiffe engine mints JWT-SVIDs, generates its own signing keys, and serves OIDC Discovery endpoints. The auth/spiffe method validates SVIDs from SPIRE or other issuers.

Aspect	Current POC (OSS)	Vault Enterprise 1.21
Trust root	SPIRE Server	Vault itself
SVID issuer	SPIRE (via spiffe-helper)	Vault SPIFFE secrets engine
OIDC Discovery	SPIRE OIDC sidecar	Vault built-in endpoints
Components	SPIRE Server + Agent + OIDC + Vault	Vault only
Attestation	Kubelet-level (SPIRE Agent)	Depends on initial auth method
License	All OSS	Enterprise required (30-day trial available)

Vault Enterprise also offers a CSI driver that mounts secrets directly into pods as files, bypassing etcd. Unlike the SPIFFE CSI driver (which mounts the Workload API socket), Vault CSI delivers the secrets themselves. Simpler, but no per-pod-label identity scoping.

9. Granular Isolation: Org to Playbook

For customers requiring isolation finer than per-org (per-project, per-playbook, per-YAML file), the architecture supports multiple approaches.

CredentialComposer: Signed Claims in the JWT

SPIRE's CredentialComposer plugin adds custom claims to JWT-SVIDs at mint time. The CEL composer (spire-credentialcomposer-cel) can derive claims from the SPIFFE ID path. These claims are cryptographically signed and cannot be tampered with by playbook authors.

AWX knows the playbook filename, project name, and organization at pod creation time (Job.playbook, Job.project.name). A small upstream change (~10 lines in receptor.py) would add

these as pod labels. SPIRE's k8s workload attestor selects on these labels, and the CredentialComposer adds them as signed JWT claims. Vault's bound_claims validates them.

Approach	Granularity	Complexity	Tamper-Proof?
SPIFFE ID per scope + CredentialComposer	Per playbook	High (one SPIRE entry + Vault role per scope)	Yes (signed JWT)
AAP Credential injection + Vault paths	Per job template	Low (AAP config only)	Yes (admin controls env var)
SPIFFE per org + Vault bound_claims	Per project	Medium	Yes (signed JWT for org, Vault policy for pro

Customer decision point: Where does the trust boundary need cryptographic enforcement, and where is policy-based enforcement sufficient? Cryptographic (SPIFFE ID per scope) gives the strongest guarantee but scales linearly with scope count. Policy-based (Vault paths + AAP credentials) is operationally simpler.

10. Demo Scenarios

Scenario 1: Platform Zero-Trust Identity

AAP's platform identity (spiffe://...ns/aap/sa/aap-controller) replaces the root Vault token. Delete the SPIRE registration entry, re-run the job, it fails immediately. Re-add the entry, it works again. No tokens to rotate, no credentials to update.

Scenario 2: Multi-Org Secret Isolation

Each AAP org gets its own SPIFFE ID and Vault policy. Infra-Corp cannot read AppDev-Squad's secrets, even though they share the same AAP instance. Verified on the live cluster.

Scenario 3: EDA Dynamic Credentials

EDA rulebook activation needs AWS credentials to remediate an incident. The EDA pod's SPIFFE identity fetches them from Vault at event time. Credentials are never persisted.

11. Customer Talking Points

Compliance and audit: 'How do you authenticate to your secrets store?' changes from 'we have a shared token' to 'each workload has a cryptographic identity scoped to its role.'

Blast radius: If one AAP org is compromised, only that org's Vault path is exposed, not the entire secrets store.

Operational: No more manual token rotation. No more 'who has the root token?' No more emergency credential invalidation affecting unrelated workloads.

Multi-tenancy: Organizations sharing the same AAP instance get cryptographically isolated secret access, not just RBAC isolation.

Standards-based: SPIFFE is a CNCF graduated project. The OIDC bridge is standards-based. No vendor lock-in; SPIRE can be replaced by any SPIFFE-compliant issuer.

12. Path to Production

Area	POC (Current)	Production Target
SPIRE	Upstream SPIRE containers	ZTWIM operator (Red Hat supported, included with OCP)
Vault storage	File backend + manual unseal	HA Raft + cloud KMS auto-unseal
Socket sharing	hostPath + spc_t SELinux	SPIFFE CSI driver (no privileged SCC)
Entry registration	Manual per-agent	ClusterSPIFFEID (ZTWIM CRD, auto-registers)
Sidecar injection	Kyverno	ZTWIM operator or Kyverno
Vault deployment	Custom StatefulSet	Vault Helm chart with HA
Playbook consumption	spiffe_vault lookup plugin (stdlib)	Ansible collection with tests + docs

The POC is running live on the hosted-1 cluster (HyperShift, KubeVirt workers). All four AAP target pods have sidecars injected, are fetching credentials from Vault via SPIFFE JWT, and are continuously renewing.